

A

AgentPay

Autonomous Payment Infrastructure for AI Agents

Judge Presentation & Live Demo Guide

Network **Solana Devnet**

Protocol **V2**

Console **127.0.0.1:3100**

ONE COMMAND. THAT IS THE WHOLE DEMO.

```
npm run demo:v2
```

*Every figure in this guide was read back from Solana devnet or recomputed independently.
Nothing here is illustrative.*

02 30-second elevator pitch

WHAT I SAY

AI agents are starting to buy things on their own — data, APIs, compute. But you cannot hand an agent your wallet and hope for the best. AgentPay is the layer in between. A human sets a spending limit. The money goes into an escrow on Solana that nothing can exceed. Every purchase the agent makes is signed, and our gateway checks it against the policy before a single request reaches the provider. Every decision — including every refusal — gets hashed into a chain and anchored on Solana. So afterwards, anyone can prove exactly what that agent spent. Without trusting us.

≈ 28 seconds at a normal speaking pace.

03 The problem

How do you let an AI agent spend money on its own without handing it unrestricted control of funds?

Today the honest answer is: you don't. You give it an API key with a credit card behind it and you hope. That leaves six concrete risks.

RISK	WHAT GOES WRONG
Unrestricted spending	An agent in a loop burns the whole balance. Nothing structurally stops it.
Replayed claims	A captured payment message is submitted twice and pays twice.
Unauthorized resources	The agent buys something nobody approved it to buy.
Gateway manipulation	Whoever runs the payment layer can quietly change amounts or destinations.
No auditability	When the bill arrives, there is no way to prove which calls were legitimate.
Unclear settlement	Who moved the money, on whose authority, and can anyone check?

The last two are the ones that matter to an auditor — and the ones nobody solves.

04 The solution

An enforcement and audit layer between an autonomous agent and the APIs it buys from.



THE ONE-LINE CLAIM

AgentPay never holds the provider's private key — and the money still lands in the provider's account, because the program binds the destination.

05 How the system works

COMPONENT	IN ONE SENTENCE
Human authorization	A person creates the agent and defines what it may spend.

Agent	An Ed25519 keypair that signs every purchase it makes.
Policy	The envelope: total cap, per-call cap, allowed resources, call count.
Escrow	A vault on Solana holding the deposit. The absolute ceiling.
Signed claim	A 73-byte message the agent signs, stating its running total.
Gateway	Verifies the claim and applies the policy <i>before</i> forwarding anything.
Provider	The API being bought from. It never handles payment logic.
Evidence chain	Every decision hashed into a chain — refusals as well as purchases.
Merkle root	One hash summarising the log, committed on chain at settlement.
Settlement authority	AgentPay's own key, which can trigger settlement and nothing else.
Verifier	A browser page that recomputes the root and compares three sources.

06 Human authorization

Two different things, often confused. Keep them separate when you speak.

Human authorization — the standing envelope

A person creates the agent, then authorizes it with a **max total**, a **max per call**, an **allowed-resources** allowlist and an optional **max calls**. Set once; the agent then operates inside it without asking anybody.

THE RULE THAT MATTERS

The envelope can only **narrow** what the escrow already permits. It never widens it.

Per-purchase approval — one decision, one spend

A separate mechanism. A spend needs an explicit human decision when the agent is in `human` mode, or is `autonomous` and the spend is at or above its `approval_threshold`. While it waits, the agent is refused with `ERR_APPROVAL_REQUIRED` — refused, not held open — and retries after the decision. Approving authorizes **that one purchase**, bound to that resource at that price. The console page is `/approvals`.

DO NOT OVERCLAIM

The canonical demo uses an autonomous agent operating inside its envelope, so it does **not** trigger a per-purchase approval. Show `/approvals` to explain the mechanism — do not say the demo exercised it.

07 V2 security model

CONTROL	HOW IT WORKS
Ed25519 signed claims	73 bytes: "agentpay:claim:v1"(17) session(32) cumulative(8) nonce(8) expires_at(8). The domain prefix stops a signature from another protocol being replayed here.
Cumulative claims	A claim states the running total, not an increment. Paying the highest pays for everything beneath it — one transfer, not one per call.
High-water mark	Only a strictly higher cumulative is accepted. Equal is a replay; lower is a regression. Both refused.
Nonce monotonicity	The sequence number must increase. An out-of-order claim is refused.
Expiry	Claims and sessions expire, with 30 s of clock-skew tolerance. Sessions cap at 30 days.
Price matching	The claimed amount must match the provider's published price for that resource.
Policy enforcement	Total, per-call, allowlist and call count — applied before anything is forwarded.
Escrow ceiling	Enforced by the program. No policy, bug or compromised gateway can exceed the deposit.
Provider destination binding	The destination is in the session PDA's seeds. It cannot change after the session opens.
Settlement authority	Written once at open_session. It can trigger a settlement and nothing else.
Evidence chain	SHA256(prev session cumulative nonce decision). Tamper-evident, refusals included.
Merkle root	Committed on chain in a SettlementRecord, so a single decision can be proved to a stranger.

Enforcement order in `/v1/buy` :

```
price → 402 → decode → expiry → session → signature
      → price match → policy → admit_claim → forward → evidence
```

The claim is admitted **before** the request is forwarded. That is why a refused claim never reaches the provider.

08 Live demo — exact commands

```
cd ~/Projects/AgentPay
```

```
docker compose ps
```

Expect four services — `gateway` , `postgres` , `provider` , `web` — all **running (healthy)**.

```
curl -s localhost:8080/health | python3 -m json.tool
```

Expect `"status": "ok"` , `"state_backend": "POSTGRES"` , and a `program_id` ending ...bwe8hTx7TpS9m.

ONE COMMAND FOR THE JUDGE

npm run demo:v2

Nothing else. Do not offer alternatives.

Runtime $\approx$ 2–3 minutes against public devnet. Spends roughly 0.02–0.05 devnet SOL in rent and fees.

09 Live demo — 9 steps

1 Run the canonical demo

WHAT I DO

```
npm run demo:v2
```

JUDGE SEES

A banner, then numbered stages scrolling past with green PASS markers.

WHAT I SAY

"One command. This runs the whole lifecycle against real Solana devnet."

2 Human authorization

WHAT I DO

Point at stages 2 and 3.

JUDGE SEES

2. a human creates an agent and binds its wallet
PASS agent created
3. the human sets the spending envelope
max_total 1000000 · max_per_call 2000 · allowed /weather, /quote
PASS envelope stored narrows the escrow, never widens it

WHAT I SAY

"A person creates the agent and sets the limit. The agent cannot widen it."

3 Bounded escrow

WHAT I DO

Point at stage 5.

JUDGE SEES

```
session PDA                6sTkEPT93VRGbLZhUXKGQdP8C9K5c1LRiwBpSFK2TvKQ
settlement_authority (chain) DLUd55GehdW6pNnv82NUs9hwbwN4mXssXwm8ucv4FH6s
PASS session account is 219 bytes    the v2 layout
```

WHAT I SAY

"The money sits in an escrow on Solana. This deposit is the absolute ceiling — no bug in my gateway can exceed it."

4 Successful signed claims

WHAT I DO

Point at stage 8.

JUDGE SEES

```
PASS bought /weather      cumulative 1000
PASS bought /quote        cumulative 1500
```

WHAT I SAY

"Every call carries a claim the agent signed. Claims are cumulative, so only the highest one ever reaches the chain."

5 Policy refusal

WHAT I DO

Point at stage 9, first refusal.

JUDGE SEES

```
PASS  /analyse refused      ERR_POLICY_RESOURCE_NOT_ALLOWED
PASS  the mark did not move  still 1500
```

WHAT I SAY

"/analyse is outside the envelope. Refused — and the provider was never contacted."

6 Replay refusal

WHAT I DO

Point at stage 9, second refusal.

JUDGE SEES

```
PASS  a reused nonce refused  ERR_NONCE_NOT_MONOTONIC
```

WHAT I SAY

"A replayed claim. The protocol refuses it, and a refusal costs nothing."

7 Evidence chain

WHAT I DO

Point at stage 10.

JUDGE SEES

```
entries      3
decisions    ALLOWED, ALLOWED, ERR_NONCE_NOT_MONOTONIC
chain_valid  true
PASS hash chain intact
```

WHAT I SAY

"Every decision is hashed into a chain — including the refusals. They are recorded, not discarded."

8 V2 settlement

WHAT I DO

Point at stage 11.

JUDGE SEES

```
provider balance after  0 -> 1500  (+1500)
vault balance after    3000000 -> 2998500  (-1500)
transaction signers     DLuD55GehdW6pNnv82NUs9hwbwN4mXssXwm8ucv4FH6s
PASS the provider key never signed
```

WHAT I SAY

"One name in that signer list: our own authority. The provider's private key does not exist on my servers — and the money still landed in the provider's account, because the program binds the destination."

WHAT I DO

Open the Verifier, click the red entry. Then point back at stage 14.

JUDGE SEES

Verified — Browser = Gateway = Chain, three identical roots. Then $1500 + 2998500 = 3000000$.

WHAT I SAY

"Recomputed in this browser. Not our word — and not mine either; that third root came off the chain. And the arithmetic closes: nothing appeared, nothing vanished."

10 Browser demo

Console: `http://127.0.0.1:3100` . Suggested order: **Overview** → **Session** → **Verifier** → **Settlement**.

ROUTE	SHOW	PRIORITY
/	Payment lifecycle strip, KPIs, Security controls	High
/session/6sTkEPT9...K2TvKQ	Escrow figures, 9-of-9 lifecycle, claim activity	High
/verifier?session=6sTkEPT9...	Three-root verification	Highest
/settle	Settlement flow, latest committed root	High
/agents	The envelope that bounds the agent	Medium
/approvals	Per-purchase human control	Medium
/registry	What is on offer, and at what price	Low
/playground	Claim simulator, no spending	Optional

11 Verifier — the main proof

This is the screen that wins the room. Open it, click the red entry, stop talking for a second.

BROWSER ROOT

MATCH

d3a6a55f...30e4e22f

computed here

GATEWAY ROOT

MATCH

d3a6a55f...30e4e22f

reported

ON-CHAIN ROOT

MATCH

d3a6a55f...30e4e22f

committed

✓ VERIFIED

- **Browser root** — recomputed from the leaf and sibling path with WebCrypto, in the judge's own browser.
- **Gateway root** — what our server claims.
- **On-chain root** — what the Solana program actually stored.

WHAT I SAY

"The browser recomputes this from scratch. It does not ask the gateway for the answer. When all three match, you have not taken anyone's word for anything."

DO NOT OVERCLAIM

This proves one decision is covered by a root committed on Solana. It does not prove the gateway is honest about decisions it never logged, and it is not an audit of the program.

12 Settlement proof

Three properties, all visible on chain: **the settlement authority signs**, **the provider's key does not**, and **the destination is bound by the session, not chosen by the signer**.

Settlement transaction	4DGEEmFK4LptHEmRRbQB5HZR8ae85BkuWsqsvJqwoXvj39fEJ29js9JhijtG3XhmXGVWXBUDwJLLyoEZ9D5uJK2aG
Settlement record	BGYMzmAF4kTbBLsbRU4GMzKHkUxwDUyjXcUMZUWp4B5H
Settlement authority (signer)	DLuD55Geh...ucv4FH6s
Provider signed	FALSE
Required signatures	1
Provider destination	2UuibK8uQy7MhUwEDaAVUb8LUuiDhK5U4KCPBPzz1xZf
Settled amount	1,500 micro-USDC (cumulative)
Remaining escrow	2,998,500 micro-USDC
V2 program present	TRUE
V1 program present	FALSE

Verify it live in front of them:

```
solana confirm -v 4DGEEmFK4LptHEmRRbQB5HZR8ae85BkuWsqsvJqwoXvj39fEJ29js9JhijtG3XhmXGVWXBUDwJLLyoEZ9D5uJK2aG -u devnet
```

13 Conservation proof

SETTLED + REMAINING = DEPOSIT

1,500 + 2,998,500 = 3,000,000

Both figures read back from Solana after settlement — the vault's token balance and the provider's token account — not from the script's own output.

WHAT I SAY

"Nothing appeared and nothing disappeared. What the provider received, plus what is still the agent's, is exactly the deposit."

The remainder is not stranded: after expiry `refund_session` is permissionless, and those funds can only ever move to the agent's own token account.

14 Failure / refusal demo

Two refusals, two different mechanisms. Both from the verified run.

ERR_POLICY_RESOURCE_NOT_ALLOWED

What happened	The agent asked for <code>/analyse</code> .
Why refused	Not in the human's allowlist (<code>/weather</code> , <code>/quote</code>).
Provider contacted	No — the claim is admitted before forwarding.
Funds moved	No — the high-water mark stayed at 1500.
Evidence recorded	Yes

This is the human's envelope doing its job.

ERR_NONCE_NOT_MONOTONIC

What happened	A claim arrived with nonce 2, already used.
Why refused	The nonce must strictly increase.
Provider contacted	No
Funds moved	No
Evidence recorded	Yes — sequence 2, cumulative 2500, nonce 2

This is the protocol refusing a replay, independent of any policy.

THE LINE WORTH SAYING

"A refusal is not an error. It is the product working — and it costs nothing, because the mark never moved."

15 V1 vs V2

	V1 — LEGACY	V2 — CANONICAL
Program	3aKGM6Cb...u7y5xP2U	ApjxJKBU...8hTx7TpS9m
Session layout	187 bytes	219 bytes
Who settles	the provider, with its own private key	the settlement authority or the provider
AgentPay holds	the provider's private key	its own authority key only
Settlement	once	monotonic, repeatable

Merkle root	final	latest committed
Role	existing sessions only	every new session and the demo

Why the judge should only see V2: V1 is the old custody model, where AgentPay holds the provider's private key — the exact thing V2 exists to remove. Demonstrating it would contradict the pitch.

⚠ NEVER RUN THESE DURING THE DEMO

```
npm run evidence:v1-legacy
npm run settle:v1-legacy
npm run policy:v1-legacy
npm run reconcile:v1-legacy
```

```
npm run sdk-demo:v1-legacy
npm run stage-settleable:v1-legacy
npm run diagnostics:v1-legacy
```

Every one drives the old program. Also avoid `npm run demo` and `npm run buy` — synthetic sessions, and they need a dev-only flag that skips on-chain reconciliation.

16 Judge questions & answers

Why does the agent need a signature?

So the amount cannot be forged by us. The program verifies the agent's Ed25519 signature over the claim, so the settled amount is fixed by the agent — not by our gateway, and not by the provider.

What prevents an agent from overspending?

Two independent limits. The escrow is a hard ceiling enforced by the program — no policy, bug or compromised gateway can exceed it. Inside that, the human's envelope caps the total, the per-call amount, and which resources are allowed.

What happens when a request is refused?

The provider is never contacted, no funds move, and the high-water mark does not advance. The refusal is still written into the evidence chain, so it is provable afterwards.

Can the provider redirect settlement?

No. The destination is derived from `session.provider`, which is inside the session PDA's seeds. It is fixed when the session opens and cannot be changed afterwards.

Who signs the V2 settlement?

AgentPay's own settlement authority — `DLuD55Geh...ucv4FH6s` in this run. Exactly one signer. The provider's key did not sign, and does not exist on our servers.

How do you prove the evidence?

Each decision is hashed into a chain, and the chain is summarised by a Merkle root committed on chain at settlement. Any one decision can be proved with an inclusion proof, recomputed in the verifier's own browser.

Is the Merkle root final?

No, and we are careful about this. V2 settlement is monotonic and repeatable, so a later settlement can commit a root over more entries. It is the **latest committed root**, not a final one. A proof exported now will not verify against a later root.

Is AgentPay audited?

No. Neither program has ever been externally audited. This is devnet only, and mainnet would need an audit first.

Why is V1 still present?

It still holds sessions opened before the cutover, and those must be able to settle or refund under the rules they were opened with. It is read-and-drain only, and is removed once every V1 session has drained.

What happens if the gateway is compromised?

It cannot manufacture an amount — that needs the agent's signature. It cannot redirect funds — the destination is in the PDA seeds. It can refuse service, and it can change a provider's *registered* address for **future** sessions. Existing sessions are safe. That residual risk is documented, not hidden.

What happens if an agent replays a claim?

Refused, two ways over. The cumulative must strictly increase, and the nonce must strictly increase. You saw it: `ERR_NONCE_NOT_MONOTONIC`.

What happens if an agent requests an unauthorized resource?

Refused before the request is forwarded. You saw it: `ERR_POLICY_RESOURCE_NOT_ALLOWED`.

17 Honest limitations

State these before a judge finds them. It reads as confidence, not weakness.

- **Not audited.** Neither program, ever. No third party has reviewed the custody model or the migration.
- **Devnet only.** Mainnet needs an audit first.
- **The root is the latest, not final.** Repeatable settlement is why.
- **V1 is still deployed** for sessions opened before the cutover, and `AGENTPAY_PROVIDER_KEYPAIR` is still configured for them. Migration is not finished.
- **The provider still needs a wallet address**, and should keep its key as a V1 fallback. The claim is that AgentPay does not *hold* it — not that it does not exist.
- **A compromised control plane can change a provider's registered settlement address for future sessions.** Existing sessions are safe; this must be disclosed to providers.
- **Evidence does not contain the resource name.** The hash preimage is session || cumulative || nonce || decision, so the console cannot show which endpoint a claim was for.
- **Rate limiting keys on the TCP peer**, so behind a load balancer all tenants share one bucket.
- **One known read-path defect:** `GET /v1/session/{s}/settlement` derives the settlement-record address from the primary program unconditionally, so a settled **V1** session is reported as unsettled. Read-only; the settle path routes correctly; it does not affect the V2 demo.
- **SettlementRecord is not in the V2 IDL** — the account became `UncheckedAccount`. Decode it by offset, as the gateway does.
- **No commercial validation.** No design partner, no pilot, no pricing research.

18 Emergency / troubleshooting

Services are not running:

```
docker compose up -d
```

The console is showing an old build:

```
docker compose up -d --build web
```

Check the gateway:

```
curl -s localhost:8080/health | python3 -m json.tool
```

Check all four services:

```
docker compose ps
```

The demo failed partway: it is safe to re-run. Each run creates a fresh agent, session and mint; it never reuses the previous one.

The console looks stale: confirm the `web` container is healthy and that you are on **:3100**. Any other port is not the deployed UI.

⚠ NEVER ON A SHARED SCREEN

Never paste a private key, an admin token, or the contents of `secrets/` into a terminal a judge can see. Nothing in this guide requires it.

19 The 60-second version

WHAT I SAY

AI agents are starting to buy things on their own, but you cannot hand one your wallet. AgentPay puts a human-defined spending limit in front of the agent, backed by an escrow on Solana that nothing can exceed. One command runs the whole thing on devnet: the agent buys two resources, gets refused twice — once by policy, once for replaying a claim — and every decision is hashed into a chain and anchored on Solana. Settlement is signed by our own key, never the provider's, and the money still lands in the provider's account because the program binds the destination. Then anyone can open a browser, recompute the evidence root themselves, and see it match the root on chain. And the arithmetic closes exactly: 1,500 settled plus 2,998,500 remaining equals the 3,000,000 deposited.

20 Closing statement

WHAT I SAY

AgentPay does not give an AI agent unrestricted access to money. A human defines the spending envelope. The agent proves every claim cryptographically. The gateway enforces the policy before a single request reaches a provider. Every decision — including every refusal — becomes tamper-evident evidence. And the settlement can be checked against the blockchain by anyone, without trusting us. It runs on Solana devnet today. It has not been audited, and we are not claiming it has been. But everything you just saw was real, and you can check every number yourself.

A Appendix — verified run data

All values read back from Solana devnet or the gateway. Verified current at the time of writing.

COMMAND	npm run demo:v2		
NETWORK	Solana devnet		
PROGRAM (V2)	ApjxJKBUUd8EEAovQe74jS9qZsRCAC2bwe8hTx7TpS9m		
V1 PRESENT	FALSE		
AGENT	"e2e v2 agent" 5vz9a8FGF6gQCSaWnK9y87CaLgKiACdRrZmWs6NsByMT autonomous · active		
ENVELOPE	max_total 1000000 · max_per_call 2000 allowed /weather, /quote · max_calls 10		
FUNDING	3000000 micro-USDC		
SESSION	6sTkePT93VRGbLZhUXKGQdP8C9K5c1LRiWbpSFK2TvKQ		
owner	ApjxJKBUUd8EEAovQe74jS9qZsRCAC2bwe8hTx7TpS9m		
length	219 bytes		
vault	F2bgLYszWdtApVwEvZ6Zgr8augXe6YD4WDEbN7SGLTf4		
provider	2UuibK8uQy7MhUwEDaAVUb8LUuiDhK5U4KCPBPzz1xZf		
authority	DLuD55GehdW6pNnv82NUs9hwbwN4mXssXwm8ucv4FH6s		
PURCHASES	/weather → cumulative 1000 /quote → cumulative 1500		
REFUSALS	ERR_POLICY_RESOURCE_NOT_ALLOWED (/analyse) ERR_NONCE_NOT_MONOTONIC (replayed nonce)		
EVIDENCE	3 entries · chain_valid true		
seq 0	ALLOWED	cumulative 1000	nonce 1
seq 1	ALLOWED	cumulative 1500	nonce 2
seq 2	ERR_NONCE_NOT_MONOTONIC	cumulative 2500	nonce 2
MERKLE ROOT	d3a6a55f1caaacbb288b8109e953b912a43e2fac0146a0995fd092e130e4e22f		
browser	MATCH		
gateway	MATCH		
on chain	MATCH		
SETTLEMENT TX	4DGEfFK4LptHEmRRbQB5HZR8ae85BkuWsqsVJqwoXvj39fEJ29js9JhiJ tG3XhmXGVWXBUDwJLLyoEZ9D5uJK2aG		
record	BGYMzMAF4kTbBLSbRU4GMzKHKUxwDUyjXcUMZUWp4BSH		
signers	DLuD55GehdW6pNnv82NUs9hwbwN4mXssXwm8ucv4FH6s (1)		
provider signed	FALSE		
settled	1500 (cumulative)		
root_may_advance	true		
CONSERVATION	1500 + 2998500 = 3000000 ✓		

Related: RUNBOOK.md · SETTLEMENT_CUSTODY.md · MIGRATION_V1_V2.md · RELEASE_V2.md · PITCH_AND_QA.md

Your one command for the judge is `npm run demo:v2`